VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science Engineering and Information Systems

BCSE302L Database Systems

Course Project

BOM-Track

A Database for Hardware Bills of Materials, Part Stock and Board Builds

Review II: Database Implementation

Submitted by	: Vishwak
Register Number	: 24BYB0053
Implementation	: PostgreSQL 16 (SQL, PL/pgSQL), Python 3
Faculty	:

October 2026

Contents

1. Implementation Overview
2. Table Creation and Constraints (DDL)
3. Data Population (DML)
4. SQL Queries and Outputs
5. Views
6. Functions and Procedures
7. Cursor
8. Triggers
9. Indexes

1. Implementation Overview

BOM-Track is implemented on PostgreSQL 16. The procedural parts are written in PL/pgSQL, which is PostgreSQL's equivalent of Oracle PL/SQL (the same ideas: stored functions, procedures, explicit cursors, exception handling and row-level triggers). A Python front end (Streamlit with the psycopg2 driver) is used for Review III. Every output in this document was produced by running the scripts listed below; nothing was typed by hand.

File	Contents
`sql/01_schema.sql`	CREATE TABLE statements with all keys and constraints
`sql/02_views.sql`	9 views that compute every derived value
`sql/03_plpgsql.sql, 03b_...`	2 functions, 3 procedures (one with a cursor), 3 triggers
`sql/04_queries.sql`	17 queries; output in logs/queries_output.log
`sql/06_indexes.sql`	Indexes on foreign-key columns
`tools/extract_kicad.py`	Reads KiCad schematics (.sch and .kicad_sch) into a flat BOM
`tools/import_boms.py, bomlib.py`	Loads BOM files into the database, one transaction per board
`tools/load_quotes.py`	Loads real supplier prices
`build_db.sh / build_db.bat`	Rebuilds the whole database from the raw files

1.1 Data Sources

All rows in the core tables come from real files. My own board is uGameBox (an AT89S52 handheld game). To get a realistic parts catalogue with parts shared between boards, I also loaded ten published open-hardware designs from GitHub. Their schematics were parsed directly, one row per reference designator.

Board	Source	Placed	Skipped
uGameBox rev 1	My KiCad BOM export (28 grouped lines)	49	0
Ubertooth One, Artichoke, Broccoli, Celery, Endive, Pogoprog, TC13 Badge	github.com/greatscottgadgets/ubertooth (KiCad 4 schematics)	431	41
OtterPill v1.3, v1.4	github.com/Jana-Marie/OtterPill	82	4
ESP32-PoE-ISO Rev N	github.com/OLIMEX/ESP32-POE-ISO (KiCad 7 schematic)	116	22

Skipped designators are mounting holes, fiducials, test pads and parts marked as not fitted (DNP, NS, NA). Nobody buys these, so they are not part of a bill of materials.

Checking the extraction: OLIMEX publishes its own BOM for Rev N. My extraction gives the same 138 designators, with none missing and none extra. For OtterPill v1.3, the published factory BOM lists 37 designators and mine has 43. The 6 extra are the pin headers J2 to J7, which the factory BOM leaves out because they are soldered by hand.

Prices come from two real sources. (a) 18 quotes from the Ubertooth Zero BOM, which lists Mouser part numbers and unit prices in USD; that file was committed on 4 November 2010, so the quotes carry that date. (b) 5 INR prices for uGameBox parts, read from the Probots and Zbotic websites on 14 October 2026 (prices include GST). A quote is linked to a part only when category, value and package all match; 10 of the 26 priced Mouser lines could not be matched and were left out rather than guessed.

2. Table Creation and Constraints (DDL)

```
-- BOM-Track: tables and constraints (PostgreSQL 16)



CREATE TABLE category (
    category_id  SERIAL       PRIMARY KEY,
    name         VARCHAR(40)  NOT NULL UNIQUE
);

CREATE TABLE part (
    part_id      SERIAL        PRIMARY KEY,
    category_id  INT           NOT NULL REFERENCES category(category_id),
    value_text   VARCHAR(160)  NOT NULL,
    footprint    VARCHAR(160)  NOT NULL DEFAULT '',
    description  VARCHAR(200),
    CONSTRAINT uq_part UNIQUE (category_id, value_text, footprint)
);

CREATE TABLE board (
    board_id     SERIAL        PRIMARY KEY,
    name         VARCHAR(60)   NOT NULL,
    revision     VARCHAR(30)   NOT NULL,
    source       VARCHAR(200),
    created_on   DATE          NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT uq_board UNIQUE (name, revision)
);

CREATE TABLE placement (
    placement_id   SERIAL       PRIMARY KEY,
    board_id       INT          NOT NULL REFERENCES board(board_id) ON DELETE CASCADE,
    part_id        INT          NOT NULL REFERENCES part(part_id),
    ref_designator VARCHAR(20)  NOT NULL,
    label          VARCHAR(40),
    CONSTRAINT uq_placement UNIQUE (board_id, ref_designator)
);

CREATE TABLE supplier (
    supplier_id  SERIAL        PRIMARY KEY,
    name         VARCHAR(60)   NOT NULL UNIQUE,
    website      VARCHAR(120)
```

```
);

CREATE TABLE price_quote (
    quote_id     SERIAL         PRIMARY KEY,
    part_id      INT            NOT NULL REFERENCES part(part_id),
    supplier_id  INT            NOT NULL REFERENCES supplier(supplier_id),
    unit_price   NUMERIC(10,4)  NOT NULL CHECK (unit_price > 0),
    currency     CHAR(3)        NOT NULL DEFAULT 'INR',
    captured_at  TIMESTAMP      NOT NULL DEFAULT now()
);

CREATE TABLE build (
    build_id     SERIAL  PRIMARY KEY,
    board_id     INT     NOT NULL REFERENCES board(board_id),
    units        INT     NOT NULL DEFAULT 1 CHECK (units > 0),
    started_on   DATE    NOT NULL DEFAULT CURRENT_DATE,
    completed_on DATE,
    CONSTRAINT ck_build_dates CHECK (completed_on IS NULL OR completed_on >= started_on)
);

CREATE TABLE stock_movement (
    movement_id   SERIAL       PRIMARY KEY,
    part_id       INT          NOT NULL REFERENCES part(part_id),
    supplier_id   INT          REFERENCES supplier(supplier_id),
    qty_delta     INT          NOT NULL CHECK (qty_delta <> 0),
    movement_type VARCHAR(10)  NOT NULL CHECK (movement_type IN
('PURCHASE','ADJUSTMENT')),
    moved_at      TIMESTAMP    NOT NULL DEFAULT now(),
    CONSTRAINT ck_purchase CHECK (movement_type <> 'PURCHASE' OR (qty_delta > 0 AND
supplier_id IS NOT NULL))
);
```

Constraints in the final schema (read from information_schema after the build):

Constraint type	Count	Examples
PRIMARY KEY	8	one surrogate key per table (SERIAL)
FOREIGN KEY	8	placement.board_id -> board (ON DELETE CASCADE), price_quote.supplier_id -> supplier
UNIQUE	5	part(category_id, value_text, footprint); placement(board_id, ref_designator); board(name, revision)
CHECK	6	unit_price > 0; units > 0; completed_on >= started_on; qty_delta <> 0; movement_type list; purchase rule
NOT NULL	31 columns	every key, name, value and date that the system depends on
DEFAULT	7 columns	currency 'INR'; captured_at now(); units 1; created_on and started_on CURRENT_DATE

3. Data Population (DML)

Rows per table after running build_db.sh (before any demonstration transactions):

Table	Rows	Where the rows come from
category	15	Assigned by the importer from the designator prefix and symbol name
part	242	Distinct (category, value, footprint) combinations across all 11 boards
board	11	uGameBox plus 10 open-hardware boards
placement	678	One row per reference designator
supplier	11	Real component distributors (Mouser, DigiKey, LCSC, Probots, Zbotic, ...)

price_quote	23	18 Mouser (USD, 2010) + 5 Probots/Zbotic (INR, 2026)
build	0	Filled by using the system (Review III)
stock_movement	0	Filled by using the system (Review III)

The build and stock_movement tables start empty on purpose. I have not bought any parts or built any boards since starting this project, so any rows here would be made up. They are filled by the demonstration transactions in Review III (11 builds and 23 stock movements after the demo script runs), and those rows are clearly labelled as demonstration data.

3.1 INSERT, UPDATE and DELETE

INSERT: the importer inserts board, part and placement rows inside one transaction per board. Planning a build is a plain INSERT:

```
INSERT INTO build(board_id, units) SELECT board_id, 1 FROM board WHERE name =
'uGameBox' RETURNING build_id, units, started_on;
 build_id | units | started_on
----------+-------+------------
        1 |     1 | 2026-10-14
(1 row)

INSERT 0 1
SELECT * FROM v_build_readiness;
 build_id |  board   | revision | units | parts_short | units_short | ready
...
(1 row)
```

UPDATE: correcting a part's description.

```
UPDATE part SET description = '8051 MCU, 8 KB flash, DIP-40' WHERE value_text =
'AT89S52' RETURNING part_id, value_text, description;
 part_id | value_text |         description
---------+------------+-----------------------------
      18 | AT89S52    | 8051 MCU, 8 KB flash, DIP-40
(1 row)

UPDATE 1
```

DELETE: a planned build can be deleted; a supplier that already has quotes cannot, because of the foreign key.

```
INSERT INTO build(board_id, units) SELECT board_id, 2 FROM board WHERE name =
'OtterPill' AND revision = 'v1.4' RETURNING build_id;
 build_id
----------
        2
(1 row)

INSERT 0 1
DELETE FROM build WHERE completed_on IS NULL AND board_id = (SELECT board_id
FROM board WHERE name = 'OtterPill' AND revision = 'v1.4') RETURNING build_id;
 build_id
----------
        2
```

```
(1 row)

DELETE 1
DELETE FROM supplier WHERE name = 'Probots';
ERROR:  update or delete on table "supplier" violates foreign key constraint
"price_quote_supplier_id_fkey" on table "price_quote"
```

4. SQL Queries and Outputs

Seventeen queries were run on the real data. Each one answers a question that comes up when working with BOMs. Long outputs are shortened; the full output is in logs/queries_output.log.

Q1. All boards with their source and number of placed components (JOIN + GROUP BY)

```
SELECT b.board_id, b.name, b.revision, COUNT(pl.placement_id) AS components
FROM board b LEFT JOIN placement pl ON pl.board_id = b.board_id
GROUP BY b.board_id ORDER BY b.board_id;
```

```
 board_id |        name         |  revision   | components
----------+---------------------+-------------+------------
        1 | uGameBox            | rev 1       |         49
        2 | Ubertooth One       | git c9dfdbd |         79
        3 | Ubertooth Artichoke | git c9dfdbd |         66
        4 | Ubertooth Broccoli  | git c9dfdbd |         79
        5 | Ubertooth Celery    | git c9dfdbd |         90
        6 | Ubertooth Endive    | git c9dfdbd |         12
        7 | Ubertooth Pogoprog  | git c9dfdbd |         10
        8 | TC13 Badge          | git c9dfdbd |         95
        9 | OtterPill           | v1.3        |         41
       10 | OtterPill           | v1.4        |         41
       11 | ESP32-PoE-ISO       | Rev N       |        116
...
(11 rows)
```

Q2. Full BOM of uGameBox, grouped like a KiCad export (view v_board_bom)

```
SELECT refs, category, value_text, qty FROM v_board_bom
WHERE board = 'uGameBox' ORDER BY category, value_text;
```

```
               refs                 |   category    |     value_text      | qty
------------------------------------+---------------+---------------------+-----
 BZ1                                | Buzzer        | Active 5V           |   1
 C1,C6,C7                           | Capacitor     | 100nF               |   3
 C2,C3                              | Capacitor     | 10uF                |   2
 C4,C5                              | Capacitor     | 33pF                |   2
 J1                                 | Connector     | 9V BATT             |   1
 J3                                 | Connector     | ISP 2x5             |   1
 J2                                 | Connector     | SW-420 (VCC/GND/DO) |   1
 Y1                                 | Crystal       | 11.0592MHz          |   1
 D10,D9                             | Diode         | 1N5819              |   2
 DS1                                | Display       | LCD 16x2 (HD44780)  |   1
 U1                                 | IC / Module   | AT89S52             |   1
...
(20 rows)
```

Q3. Placements that carry a label different from the part value (uGameBox buttons)

```
SELECT pl.ref_designator, pl.label, p.value_text
```

```
FROM placement pl JOIN part p ON p.part_id = pl.part_id
WHERE pl.label IS NOT NULL ORDER BY pl.ref_designator;
```

```
 ref_designator | label |   value_text
----------------+-------+----------------
 SW1            | BTN0  | Push button 6mm
 SW2            | BTN1  | Push button 6mm
 SW3            | BTN2  | Push button 6mm
 SW4            | BTN3  | Push button 6mm
 SW5            | BTN4  | Push button 6mm
 SW6            | BTN5  | Push button 6mm
 SW7            | BTN6  | Push button 6mm
 SW8            | BTN7  | Push button 6mm
 SW9            | RESET | Push button 6mm
(9 rows)
```

Q4. Number of distinct parts and placements per category (aggregation)

```
SELECT c.name AS category, COUNT(DISTINCT p.part_id) AS distinct_parts,
COUNT(pl.placement_id) AS placements
FROM category c JOIN part p ON p.category_id = c.category_id
LEFT JOIN placement pl ON pl.part_id = p.part_id
GROUP BY c.name ORDER BY placements DESC;
```

```
      category      | distinct_parts | placements
--------------------+----------------+------------
 Capacitor          |             55 |        206
 Resistor           |             45 |        199
 LED                |             28 |         79
 Connector          |             36 |         64
 IC / Module        |             29 |         40
 Inductor / Ferrite |             17 |         25
 Diode              |              9 |         18
 Switch             |              6 |         18
 Transistor         |              6 |         14
 Crystal            |              6 |          8
 Battery            |              1 |          3
...
(15 rows)
```

Q5. Most reused parts across all boards (v_shared_parts, top 10)

```
SELECT category, value_text, footprint, boards, placements
FROM v_shared_parts ORDER BY boards DESC, placements DESC LIMIT 10;
```

```
 category  | value_text |      footprint      | boards | placements
-----------+------------+---------------------+--------+------------
 Resistor  | 330        | 0603                |      7 |         46
 Capacitor | 100nF      | 0603                |      6 |         37
 Resistor  | 10k        | 0603                |      5 |         21
 Capacitor | 1µF        | 0603                |      5 |         15
 Resistor  | 33         | 0603                |      5 |         10
 LED       | RSTLED     | 0603D               |      5 |          5
 Connector | JTAG       | 50MIL-HEADER-2X5-TH |      5 |          5
 LED       | USBLED     | 0603D               |      5 |          5
 Connector | EXPAND     | PIN_ARRAY_3X2       |      5 |          5
 Resistor  | 1k5        | 0603                |      5 |          5
(10 rows)
```

Q6. Values on uGameBox that other boards also use, but in a different package (subquery with IN)

```
SELECT c.name AS category, p.value_text AS value_on_ugamebox,
       (SELECT COUNT(DISTINCT pl2.board_id) FROM placement pl2 JOIN part p2 ON p2.part_id
= pl2.part_id
        WHERE p2.category_id = p.category_id AND p2.value_text = p.value_text AND
p2.part_id <> p.part_id) AS other_boards
FROM part p JOIN category c USING (category_id)
WHERE p.part_id IN (SELECT pl.part_id FROM placement pl JOIN board b USING (board_id)
WHERE b.name = 'uGameBox')
  AND EXISTS (SELECT 1 FROM part p2 WHERE p2.category_id = p.category_id AND
p2.value_text = p.value_text AND p2.part_id <> p.part_id)
ORDER BY other_boards DESC;
```

```
 category  | value_on_ugamebox | other_boards
-----------+-------------------+--------------
 Capacitor | 100nF             |            6
 Resistor  | 10k               |            5
 Capacitor | 33pF              |            1
 Resistor  | 220               |            1
(4 rows)
```

uGameBox is all through-hole, while the other boards are surface-mount. So no exact part is shared, but the same values (100nF, 10k) appear in other packages. This is why a part is defined by value and footprint together.

Q7. Parts that are on uGameBox and on no other board (NOT EXISTS)

```
SELECT p.value_text, c.name AS category FROM part p JOIN category c USING (category_id)
WHERE EXISTS (SELECT 1 FROM placement pl JOIN board b USING (board_id) WHERE pl.part_id =
p.part_id AND b.name = 'uGameBox')
  AND NOT EXISTS (SELECT 1 FROM placement pl JOIN board b USING (board_id) WHERE
pl.part_id = p.part_id AND b.name <> 'uGameBox')
ORDER BY 2, 1;
```

```
     value_text      |   category
---------------------+---------------
 Active 5V           | Buzzer
 100nF               | Capacitor
 10uF                | Capacitor
 33pF                | Capacitor
 9V BATT             | Connector
 ISP 2x5             | Connector
 SW-420 (VCC/GND/DO) | Connector
 11.0592MHz          | Crystal
 1N5819              | Diode
 LCD 16x2 (HD44780)  | Display
 AT89S52             | IC / Module
...
(20 rows)
```

Q8. Boards bigger than the average board (HAVING with a scalar subquery)

```
SELECT b.name, b.revision, COUNT(*) AS components FROM board b JOIN placement pl USING
(board_id)
GROUP BY b.board_id HAVING COUNT(*) > (SELECT AVG(n) FROM (SELECT COUNT(*) AS n FROM
placement GROUP BY board_id) t)
ORDER BY components DESC;
```

```
        name         |  revision   | components
---------------------+-------------+------------
 ESP32-PoE-ISO       | Rev N       |        116
 TC13 Badge          | git c9dfdbd |         95
 Ubertooth Celery    | git c9dfdbd |         90
 Ubertooth Broccoli  | git c9dfdbd |         79
 Ubertooth One       | git c9dfdbd |         79
 Ubertooth Artichoke | git c9dfdbd |         66
(6 rows)
```

Q9. Board cost from latest quotes, with price coverage (v_board_cost)

```
SELECT board, revision, currency, priced_cost, priced_lines || ' of ' || total_lines AS
lines_priced
FROM v_board_cost ORDER BY currency, priced_cost DESC;
```

```
        board        |  revision   | currency | priced_cost | lines_priced
---------------------+-------------+----------+-------------+--------------
 uGameBox            | rev 1       | INR      |      315.84 | 4 of 20
 Ubertooth One       | git c9dfdbd | USD      |       11.47 | 14 of 49
 Ubertooth Celery    | git c9dfdbd | USD      |       10.61 | 8 of 56
 TC13 Badge          | git c9dfdbd | USD      |       10.19 | 12 of 57
 Ubertooth Artichoke | git c9dfdbd | USD      |        1.12 | 9 of 37
 Ubertooth Broccoli  | git c9dfdbd | USD      |        1.03 | 6 of 46
 Ubertooth Pogoprog  | git c9dfdbd | USD      |        0.12 | 2 of 8
 Ubertooth Endive    | git c9dfdbd | USD      |        0.09 | 1 of 10
(8 rows)
```

Only priced lines are added up, so the coverage column is shown next to the cost. A board cost with 4 of 20 lines priced is not the full cost, and the query makes that visible.

Q10. Cost of uGameBox through function fn_board_cost

```
SELECT fn_board_cost((SELECT board_id FROM board WHERE name = 'uGameBox'), 'INR') AS
ugamebox_cost_inr;
```

```
 ugamebox_cost_inr
-------------------
            315.84
(1 row)
```

Q11. Supplier comparison: number of quotes and average unit price (OUTER JOIN)

```
SELECT s.name, COUNT(q.quote_id) AS quotes, ROUND(AVG(q.unit_price), 2) AS avg_price,
MIN(q.currency) AS currency
FROM supplier s LEFT JOIN price_quote q ON q.supplier_id = s.supplier_id
GROUP BY s.name ORDER BY quotes DESC, s.name;
```

```
        name         | quotes | avg_price | currency
---------------------+--------+-----------+----------
 Mouser Electronics  |     18 |      1.13 | USD
 Probots             |      3 |     94.33 | INR
 Zbotic              |      2 |     70.92 | INR
 DigiKey             |      0 |           |
 Evelta              |      0 |           |
 Farnell / element14 |      0 |           |
 LCSC Electronics    |      0 |           |
 Robocraze           |      0 |           |
 Robu.in             |      0 |           |
 Sunrom Electronics  |      0 |           |
```

```
 ThinkRobotics       |      0 |           |
...
(11 rows)
```

Q12. Supplier ranking per part: cheapest quote first (window function RANK)

```
SELECT p.value_text, s.name AS supplier, q.unit_price, q.currency,
       RANK() OVER (PARTITION BY q.part_id, q.currency ORDER BY q.unit_price) AS
price_rank
FROM price_quote q JOIN part p ON p.part_id = q.part_id JOIN supplier s ON s.supplier_id
= q.supplier_id
WHERE q.currency = 'INR'
ORDER BY p.value_text, price_rank;
```

```
     value_text      | supplier | unit_price | currency | price_rank
---------------------+----------+------------+----------+------------
 11.0592MHz          | Probots  |    15.0000 | INR      |          1
 AT89S52             | Zbotic   |   103.8400 | INR      |          1
 AT89S52             | Probots  |   109.0000 | INR      |          2
 LCD 16x2 (HD44780)  | Probots  |   159.0000 | INR      |          1
 SW-420 (VCC/GND/DO) | Zbotic   |    38.0000 | INR      |          1
(5 rows)
```

Q13. Unpriced lines of uGameBox (anti join)

```
SELECT bb.refs, bb.category, bb.value_text FROM v_board_bom bb
LEFT JOIN v_latest_quote lq ON lq.part_id = bb.part_id
WHERE bb.board = 'uGameBox' AND lq.part_id IS NULL ORDER BY bb.category;
```

```
                refs                 |   category    |   value_text
-------------------------------------+---------------+-----------------
 BZ1                                 | Buzzer        | Active 5V
 C1,C6,C7                            | Capacitor     | 100nF
 C2,C3                               | Capacitor     | 10uF
 C4,C5                               | Capacitor     | 33pF
 J3                                  | Connector     | ISP 2x5
 J1                                  | Connector     | 9V BATT
 D10,D9                              | Diode         | 1N5819
 U2                                  | IC / Module   | L7805
 D1,D2,D3,D4,D5,D6,D7,D8             | LED           | LED
 RV1                                 | Potentiometer | 10k
 R12,R9                              | Resistor      | 10k
...
(16 rows)
```

This list is what still needs a price before the full uGameBox cost can be known.

Q14. Capacitor values used on the Ubertooth family, by package (string functions + CASE)

```
SELECT p.footprint AS package, COUNT(DISTINCT p.value_text) AS distinct_values,
       CASE WHEN COUNT(*) > 50 THEN 'heavy' WHEN COUNT(*) > 10 THEN 'medium' ELSE 'light'
END AS usage
FROM placement pl JOIN part p USING (part_id) JOIN category c USING (category_id) JOIN
board b USING (board_id)
WHERE c.name = 'Capacitor' AND b.source LIKE '%ubertooth%'
GROUP BY p.footprint ORDER BY COUNT(*) DESC;
```

```
      package       | distinct_values | usage
--------------------+-----------------+--------
 0603               |              18 | heavy
```

```
 0402               |             11 | medium
                    |              5 | light
 GSG-1206           |              1 | light
 GSG-UUR1C101MCL1GS |              1 | light
 SM1206POL          |              1 | light
(6 rows)
```

Q15. Pairs of boards that share the most parts (self join)

```
SELECT a.name || ' ' || a.revision AS board_a, b.name || ' ' || b.revision AS board_b,
COUNT(*) AS common_parts
FROM (SELECT DISTINCT board_id, part_id FROM placement) x
JOIN (SELECT DISTINCT board_id, part_id FROM placement) y ON y.part_id = x.part_id AND
x.board_id < y.board_id
JOIN board a ON a.board_id = x.board_id JOIN board b ON b.board_id = y.board_id
GROUP BY a.name, a.revision, b.name, b.revision ORDER BY common_parts DESC LIMIT 8;
```

```
             board_a              |             board_b              | common_parts
----------------------------------+----------------------------------+--------------
 Ubertooth Broccoli git c9dfdbd  | Ubertooth Celery git c9dfdbd    |           39
 Ubertooth Celery git c9dfdbd    | TC13 Badge git c9dfdbd          |           33
 Ubertooth Broccoli git c9dfdbd  | TC13 Badge git c9dfdbd          |           32
 Ubertooth One git c9dfdbd       | Ubertooth Artichoke git c9dfdbd |           26
 OtterPill v1.3                  | OtterPill v1.4                  |           24
 Ubertooth One git c9dfdbd       | TC13 Badge git c9dfdbd          |           19
 Ubertooth One git c9dfdbd       | Ubertooth Celery git c9dfdbd    |           15
 Ubertooth Artichoke git c9dfdbd | TC13 Badge git c9dfdbd          |           14
(8 rows)
```

The Ubertooth family boards share many parts, which is the situation where a shared parts catalogue saves work. OtterPill v1.3 and v1.4 share all 24 parts; the two revisions differ only in PCB layout.

Q16. Where-used: every board and designator that uses a 100nF 0402 capacitor (correlated lookup)

```
SELECT b.name, b.revision, STRING_AGG(pl.ref_designator, ',' ORDER BY pl.ref_designator)
AS refs
FROM placement pl JOIN board b USING (board_id) JOIN part p USING (part_id)
WHERE p.value_text IN ('100nF','100n','0.1uF','100n 50V') AND p.footprint ILIKE '%0402%'
GROUP BY b.name, b.revision ORDER BY b.name;
```

```
        name         |  revision   |           refs
---------------------+-------------+--------------------------
 OtterPill           | v1.3        | C10,C12,C3,C5,C6,C7,C8,C9
 OtterPill           | v1.4        | C10,C12,C3,C5,C6,C7,C8,C9
 TC13 Badge          | git c9dfdbd | C17,C19,C3,C7
 Ubertooth Artichoke | git c9dfdbd | C1,C22,C4
 Ubertooth One       | git c9dfdbd | C16,C24,C25
(5 rows)
```

Q17. Number of placements per designator prefix (regular expressions)

```
SELECT SUBSTRING(ref_designator FROM '^[A-Za-z_-]+') AS prefix, COUNT(*) AS n
FROM placement GROUP BY 1 HAVING COUNT(*) >= 5 ORDER BY n DESC;
```

```
 prefix |  n
--------+-----
```

```
 C      | 206
 R      | 196
 D      |  92
 U      |  39
 P      |  33
 J      |  26
 L      |  20
 SW     |  16
 Q      |  11
 X      |   7
 FB     |   5
...
(11 rows)
```

5. Views

Every derived value is a view. No total, count or status is stored in a table.

View	What it computes
v_board_bom	Quantity of each part on each board (COUNT of placements), with the designator list
v_part_consumed	Parts used by completed builds: units x placements
v_stock_on_hand	SUM(qty_delta) of movements minus consumption
v_latest_quote	Most recent quote per part and currency (DISTINCT ON)
v_board_cost	BOM quantity x latest price, with how many lines are priced
v_build_status	IN PROGRESS / COMPLETED from the build dates
v_build_shortfall	Needed vs on hand for every part of every open build
v_build_readiness	Parts short per open build; ready when none are short
v_shared_parts	Parts placed on more than one board

6. Functions and Procedures

6.1 Function fn_stock_on_hand

Returns the stock on hand of one part; raises an error for a part that does not exist. Used by the stock trigger and the app.

```
CREATE OR REPLACE FUNCTION fn_stock_on_hand(p_part INT) RETURNS INT
LANGUAGE plpgsql STABLE AS $$
DECLARE v INT;
BEGIN
    SELECT on_hand INTO v FROM v_stock_on_hand WHERE part_id = p_part;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'part % does not exist', p_part;
    END IF;
    RETURN v;
END $$;
```

6.2 Function fn_board_cost

Returns the cost of one board in a given currency from the latest quotes. Example (Q10): fn_board_cost(1, 'INR') = 315.84 for uGameBox, which is 103.84 + 159 + 15 + 38 for the four priced lines.

```
CREATE OR REPLACE FUNCTION fn_board_cost(p_board INT, p_currency CHAR(3) DEFAULT 'INR')
RETURNS NUMERIC
LANGUAGE plpgsql STABLE AS $$
DECLARE v NUMERIC;
BEGIN
    SELECT COALESCE(SUM(bb.qty * lq.unit_price), 0) INTO v
    FROM v_board_bom bb JOIN v_latest_quote lq ON lq.part_id = bb.part_id AND lq.currency
= p_currency
    WHERE bb.board_id = p_board;
    RETURN ROUND(v, 2);
END $$;
```

6.3 Procedure sp_record_purchase

Records parts coming in. If a price is given it is stored as a quote too, so prices live only in price_quote.

```
CREATE OR REPLACE PROCEDURE sp_record_purchase(p_part INT, p_supplier INT, p_qty INT,
                                               p_unit_price NUMERIC DEFAULT NULL,
p_currency CHAR(3) DEFAULT 'INR')
LANGUAGE plpgsql AS $$
BEGIN
    IF p_qty <= 0 THEN
        RAISE EXCEPTION 'purchase quantity must be positive, got %', p_qty;
    END IF;
    INSERT INTO stock_movement(part_id, supplier_id, qty_delta, movement_type)
    VALUES (p_part, p_supplier, p_qty, 'PURCHASE');
    IF p_unit_price IS NOT NULL THEN
        INSERT INTO price_quote(part_id, supplier_id, unit_price, currency)
        VALUES (p_part, p_supplier, p_unit_price, p_currency);
    END IF;
END $$;
```

6.4 Procedure sp_complete_build

Marks a build as completed. It checks that the build exists and is still open; the stock check is done by the trigger trg_build_complete so that it also applies to any direct UPDATE.

```
CREATE OR REPLACE PROCEDURE sp_complete_build(p_build INT)
LANGUAGE plpgsql AS $$
DECLARE v_done DATE;
BEGIN
    SELECT completed_on INTO v_done FROM build WHERE build_id = p_build;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'build % does not exist', p_build;
    ELSIF v_done IS NOT NULL THEN
        RAISE EXCEPTION 'build % was already completed on %', p_build, v_done;
    END IF;
    UPDATE build SET completed_on = CURRENT_DATE WHERE build_id = p_build;
END $$;
```

7. Cursor

sp_shopping_list opens an explicit cursor over the shortfall of one build, fetches the rows one at a time, looks up the cheapest latest quote for each part, prints one line per part and returns the total cost and the number of unpriced parts through INOUT parameters.

```
CREATE OR REPLACE PROCEDURE sp_shopping_list(p_build INT, p_currency CHAR(3), INOUT
p_total NUMERIC DEFAULT 0,
                                             INOUT p_unpriced INT DEFAULT 0)
LANGUAGE plpgsql AS $$
DECLARE
    cur_short CURSOR FOR
        SELECT sf.part_id, sf.category, sf.value_text, sf.shortfall
        FROM v_build_shortfall sf
        WHERE sf.build_id = p_build AND sf.shortfall > 0
        ORDER BY sf.category, sf.value_text;
    r         RECORD;
    v_price   NUMERIC;
    v_sup     TEXT;
BEGIN
    p_total := 0; p_unpriced := 0;
    OPEN cur_short;
    LOOP
        FETCH cur_short INTO r;
        EXIT WHEN NOT FOUND;
        SELECT lq.unit_price, lq.supplier INTO v_price, v_sup
        FROM v_latest_quote lq WHERE lq.part_id = r.part_id AND lq.currency = p_currency;
        IF FOUND THEN
            p_total := p_total + v_price * r.shortfall;
            RAISE NOTICE '% x % % from % @ % = %', r.shortfall, r.category, r.value_text,
v_sup, v_price, v_price * r.shortfall;
        ELSE
            p_unpriced := p_unpriced + 1;
            RAISE NOTICE '% x % % : no % quote', r.shortfall, r.category, r.value_text,
p_currency;
        END IF;
    END LOOP;
    CLOSE cur_short;
    p_total := ROUND(p_total, 2);
END $$;
```

Output for the uGameBox build (from logs/transactions_demo.log):

```
CALL sp_shopping_list(1, 'INR', NULL, NULL);
NOTICE:  1 x Buzzer Active 5V : no INR quote
NOTICE:  3 x Capacitor 100nF : no INR quote
NOTICE:  2 x Capacitor 10uF : no INR quote
NOTICE:  2 x Capacitor 33pF : no INR quote
NOTICE:  1 x Connector 9V BATT : no INR quote
NOTICE:  1 x Connector ISP 2x5 : no INR quote
NOTICE:  1 x Connector SW-420 (VCC/GND/DO) from Zbotic @ 38.0000 = 38.0000
NOTICE:  1 x Crystal 11.0592MHz from Probots @ 15.0000 = 15.0000
NOTICE:  2 x Diode 1N5819 : no INR quote
NOTICE:  1 x Display LCD 16x2 (HD44780) from Probots @ 159.0000 = 159.0000
NOTICE:  1 x IC / Module AT89S52 from Zbotic @ 103.8400 = 103.8400
NOTICE:  1 x IC / Module L7805 : no INR quote
NOTICE:  8 x LED LED : no INR quote
NOTICE:  1 x Potentiometer 10k : no INR quote
NOTICE:  2 x Resistor 10k : no INR quote
NOTICE:  9 x Resistor 1k : no INR quote
NOTICE:  1 x Resistor 220 : no INR quote
NOTICE:  9 x Switch Push button 6mm : no INR quote
NOTICE:  1 x Switch SLIDE SPDT : no INR quote
NOTICE:  1 x Transistor BC547 : no INR quote
 p_total | p_unpriced
---------+------------
  315.84 |         16
(1 row)
```

8. Triggers

8.1 trg_stock_guard (BEFORE INSERT on stock_movement)

Rejects a negative adjustment that would take stock below zero. It locks the part row first so two adjustments of the same part are checked one after the other.

```
CREATE OR REPLACE FUNCTION trg_fn_stock_guard() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE v_on_hand INT;
BEGIN
    IF NEW.qty_delta < 0 THEN
        PERFORM 1 FROM part WHERE part_id = NEW.part_id FOR UPDATE;   -- serialize stock
changes per part
        v_on_hand := fn_stock_on_hand(NEW.part_id);
        IF v_on_hand + NEW.qty_delta < 0 THEN
            RAISE EXCEPTION 'stock for part % would go negative (on hand %, change %)',
                            NEW.part_id, v_on_hand, NEW.qty_delta;
        END IF;
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER trg_stock_guard BEFORE INSERT ON stock_movement
FOR EACH ROW EXECUTE FUNCTION trg_fn_stock_guard();
```

```
INSERT INTO stock_movement(part_id, qty_delta, movement_type)
SELECT part_id, -1, 'ADJUSTMENT' FROM placement WHERE board_id = 1 AND ref_designator = 'D1';
ERROR:  stock for part 5 would go negative (on hand 0, change -1)
```

8.2 trg_build_complete (BEFORE UPDATE OF completed_on on build)

A build can only be completed when no part is short. The parts of the board are locked with SELECT ... FOR UPDATE, which matters when two builds are completed at the same time (see Review III, concurrency).

```
-- T2: a build can only be completed if every part is in stock
CREATE OR REPLACE FUNCTION trg_fn_build_complete() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE r RECORD; v_short BOOLEAN;
BEGIN
    IF OLD.completed_on IS NULL AND NEW.completed_on IS NOT NULL THEN
        -- lock the parts of this board so two builds cannot use the same stock at once
        PERFORM 1 FROM part WHERE part_id IN (SELECT part_id FROM placement WHERE
board_id = NEW.board_id)
        ORDER BY part_id FOR UPDATE;
        SELECT * INTO r FROM v_build_shortfall WHERE build_id = NEW.build_id AND
shortfall > 0
        ORDER BY shortfall DESC LIMIT 1;
        v_short := FOUND;
        -- optional delay, only set by the concurrency demo, to widen the timing window
        IF current_setting('bomtrack.demo_delay', true) IS NOT NULL AND
current_setting('bomtrack.demo_delay', true) <> '' THEN
            PERFORM pg_sleep(current_setting('bomtrack.demo_delay', true)::NUMERIC);
        END IF;
        IF v_short THEN
            RAISE EXCEPTION 'build % cannot be completed: short of % x % % (and possibly
more)',
                            NEW.build_id, r.shortfall, r.category, r.value_text;
        END IF;
    END IF;
```

```
    RETURN NEW;
END $$;
DROP TRIGGER IF EXISTS trg_build_complete ON build;
CREATE TRIGGER trg_build_complete BEFORE UPDATE OF completed_on ON build
FOR EACH ROW EXECUTE FUNCTION trg_fn_build_complete();
```

```
CALL sp_complete_build(1);
ERROR:  build 1 cannot be completed: short of 9 x Resistor 1k (and possibly more)
SELECT * FROM v_build_status;
 build_id |  board   | revision | units | started_on | completed_on |   status
----------+----------+----------+-------+------------+--------------+-------------
        1 | uGameBox | rev 1    |     1 | 2026-10-14 |              | IN PROGRESS
(1 row)
```

8.3 trg_freeze_bom (BEFORE INSERT/UPDATE/DELETE on placement)

Past consumption is computed from a board's placements, so editing the BOM of a board that has been built would silently change history. The trigger blocks this; a design change must be stored as a new revision.

```
CREATE OR REPLACE FUNCTION trg_fn_freeze_bom() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE v_board INT := COALESCE(NEW.board_id, OLD.board_id);
BEGIN
    IF EXISTS (SELECT 1 FROM build WHERE board_id = v_board AND completed_on IS NOT NULL)
THEN
        RAISE EXCEPTION 'board % has completed builds; create a new revision instead of
editing its BOM', v_board;
    END IF;
    RETURN COALESCE(NEW, OLD);
END $$;
CREATE TRIGGER trg_freeze_bom BEFORE INSERT OR UPDATE OR DELETE ON placement
FOR EACH ROW EXECUTE FUNCTION trg_fn_freeze_bom();
```

```
DELETE FROM placement WHERE board_id = 1 AND ref_designator = 'R11';
ERROR:  board 1 has completed builds; create a new revision instead of editing its BOM
UPDATE placement SET label = 'FIRE' WHERE board_id = 1 AND ref_designator = 'SW1';
ERROR:  board 1 has completed builds; create a new revision instead of editing its BOM
```

9. Indexes

PostgreSQL creates indexes for primary keys and UNIQUE constraints automatically, but not for foreign keys. I added indexes on the foreign-key columns used in the views' joins:

```
-- Foreign keys are not indexed automatically in PostgreSQL. These support the joins used
by the views.
CREATE INDEX IF NOT EXISTS idx_placement_part   ON placement(part_id);
CREATE INDEX IF NOT EXISTS idx_quote_part       ON price_quote(part_id, currency,
captured_at DESC);
CREATE INDEX IF NOT EXISTS idx_movement_part    ON stock_movement(part_id);
CREATE INDEX IF NOT EXISTS idx_build_board      ON build(board_id);
```

Their effect on the real data (678 placements in 4 pages) is measured and discussed honestly in Review III.